



Parcours AI Engineer

SOUTENANCE PROJET 10

“Développer un système de recommandation d’articles - My Content”

Stéphanie Duhem - Juillet 2026



00. CONTEXTE & DÉROULÉ DU PROJET

LE CONTEXTE

- **My Content** : recommandation de lecture ➡ développer un **premier MVP** de recommandation d'articles.
- **User story** : "En tant qu'utilisateur, je vais recevoir une sélection de 5 articles."
- **Données** : jeu de données public Globocom (Kaggle) — interactions utilisateurs / articles
- **Contrainte technique** : architecture serverless sur Azure

LES GRANDES ÉTAPES DU PROJET

- Analyse exploratoire des données et des embeddings d'articles
- Test de deux approches : *Content-Based Filtering* et *Collaborative Filtering*
- Déploiement de la meilleure approche serverless avec *Azure Function*
- Réflexion sur l'architecture cible pour nouveaux utilisateurs et articles

01-A. DATASET

GLOBOCOM – NEWS PORTAL USER INTERACTIONS (KAGGLE)

322 897

utilisateurs

46 033

articles cliqués

364 047

articles au total

250

dimensions d'embedding

CONCLUSIONS DE L'EXPLORATION DES DONNÉES

- Distribution très asymétrique des clics par utilisateur : médiane **4 clics**, moyenne **9,25 clics**, max **1 232 clics**
- Matrice user × article très creuse (sparse) : **99,98 % de zéros**
- Embeddings bruts : 364 047 × 250 dims > **364 Mo**
- Les **métadonnées** (catégorie, nombre de mots, date) > enrichissement de l'affichage

Problème spécifique à ce dataset :

- **Cold-start sévère** : 25 % des utilisateurs $\leq$ 2 clics
- **Cold-start utilisateur** : nouvel utilisateur sans historique > stratégie du *fallback*
- **Cold-start article** : nouvel article jamais cliqué > géré différemment selon l'approche (*cf. comparaison des modèles*)

01-B. PANORAMA DES STRATÉGIES DE RECOMMANDATION

Stratégie	Principe	Exemple
Content-Based Filtering	Similarité entre items (contenu, embeddings)	Articles → similarité cosinus Linkedin (jobs) Pandora (Music) IMDb
Collaborative Filtering	Comportements collectifs (qui a aimé quoi)	ALS, SVD, KNN sur matrice user × item Amazon (CF historique)
Hybride	Combinaison de plusieurs signaux	Netflix, Spotify, YouTube
Popularité / Editorial	Top articles, curation humaine	Fallback cold-start Reddit (tri par votes) App Store, Le Monde
Contextuel	Heure, localisation, appareil, saisonnalité	Publicité ciblée, recherche locale Uber Eats, Booking.com

Exemple Netflix — stratégie multi-niveau

- Par **région** : pondération différente selon les marchés (coréens → K-dramas, France → films d'auteur)
- Par **moment** : recommandations différentes le soir VS le week-end
- Par **profil** : un même compte avec plusieurs profils → modèles séparés
- **Hybridation** : CF (comportements) + CB (similarité de contenu) + éditorial (mise en avant de nouveautés)

01-C. DEUX APPROCHES DE RECOMMANDATION TESTÉES

CONTENT-BASED FILTERING

- Recommander des **articles similaires** à ceux déjà lus
- Comparaison du contenu textuel via les **embeddings**
- **ACP** appliquée : 250 > 50 dimensions — **variance conservée à 94,53 % & 80% de gain mémoire** (364 Mo > 72,8 Mo)
- **Métrique** : similarité cosinus entre vecteurs d'articles
- **Implémentation sans algorithme** : approche directe (embeddings > PCA > similarité cosinus)

COLLABORATIVE FILTERING (ALS)

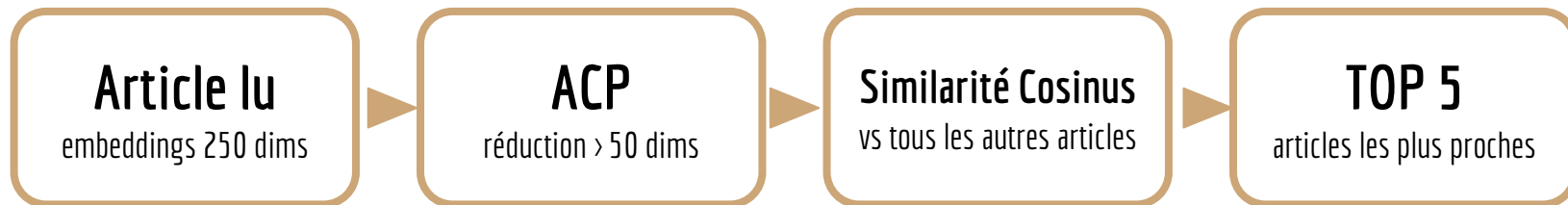
- Recommander des articles lus par des **users similaires**
- Aucune analyse du contenu — basé uniquement sur les **clics**
- **Algorithme** : **ALS** (Alternating Least Squares, library implicit) adapté au **feedback implicite** & optimisé pour les **matrices creuses**
- **Métrique** : produit scalaire (dot product) entre vecteur utilisateur et vecteurs articles

ÉVALUATION COMMUNE

- Protocole **leave-one-out** sur **500** utilisateurs test : on masque le dernier article lu de chaque utilisateur test
- On vérifie si cet article apparaît dans le top des 5 recommandations > score **Hit Rate@5** ( *métrique stricte*)

02-A. CONTENT-BASED FILTERING – PRINCIPE

PIPELINE



AVANTAGES

- Fonctionne dès 1 seul clic
- Cold-start article géré : un nouvel article dispose immédiatement de son embedding > recommandable sans aucun clic préalable
- Résultats interprétables (similarité sémantique)
- Pas de réentraînement pour nouveaux articles ni nouveaux utilisateurs

LIMITES

- N'exploite pas les comportements collectifs
- Risque de bulle de filtre (articles trop similaires)
- Dépend de la qualité des embeddings textuels

02-B. CONTENT-BASED FILTERING – RÉSULTATS

SCORE OBTENU

0,40%

Hit Rate@5

~ 5 min

temps d'évaluation

72.8 Mo

taille de l'artifact

94,53%

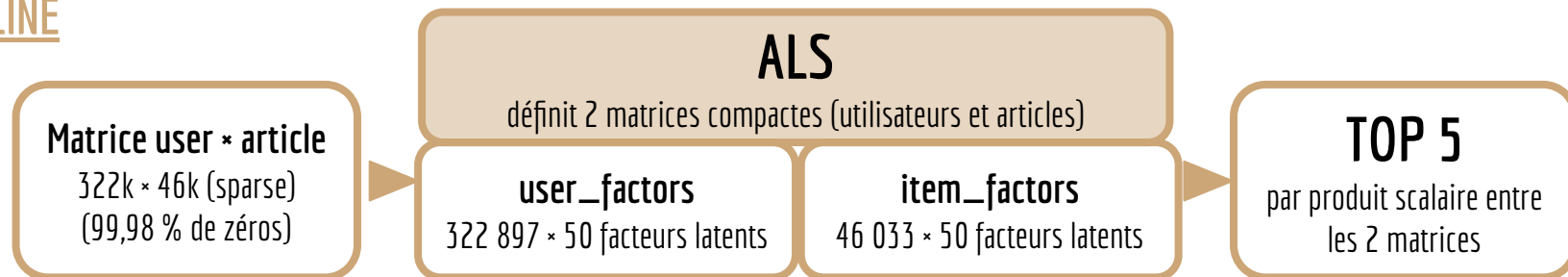
variance ACP conservée

ANALYSE

- Sur 500 utilisateurs testés, l'article masqué n'apparaît que très rarement dans le Top 5, seulement dans **0,4%** des cas
- La similarité cosinus capture la proximité sémantique mais pas les préférences individuelles
- Deux articles de même catégorie peuvent être recommandés même si l'utilisateur ne les aurait pas choisis
- Le leave-one-out est strict : un bon article recommandé qui n'est pas exactement l'article masqué n'est pas comptabilisé comme une bonne recommandation.
- Temps d'évaluation très **rapide**
- **Faible taille** de la matrice d'embeddings réduits grâce à l'ACP

03-A. COLLABORATIVE FILTERING (ALS) – PRINCIPE

PIPELINE



AVANTAGES

- Exploite les comportements des 322 897 utilisateurs
- Capture des **profils de goût** complexes
- Ne nécessite pas le contenu des articles
- **Efficace** sur données implicites (clics)

LIMITES

- **Cold-start user** : contourné par **fallback**
- **Cold-start article** : le CF ne peut pas le recommander (contrairement au Content-Based)
- Faible interprétabilité (facteurs latents, abstraits > effet boîte noire)
- Réentraînement complet obligatoire si ajout de nouveaux articles ou utilisateurs

03-B. COLLABORATIVE FILTERING (ALS) – RÉSULTATS

SCORE OBTENU

10,60%

HitRate@5

~ 355 min

temps d'évaluation

119.1 Mo

taille total des artifacts

50

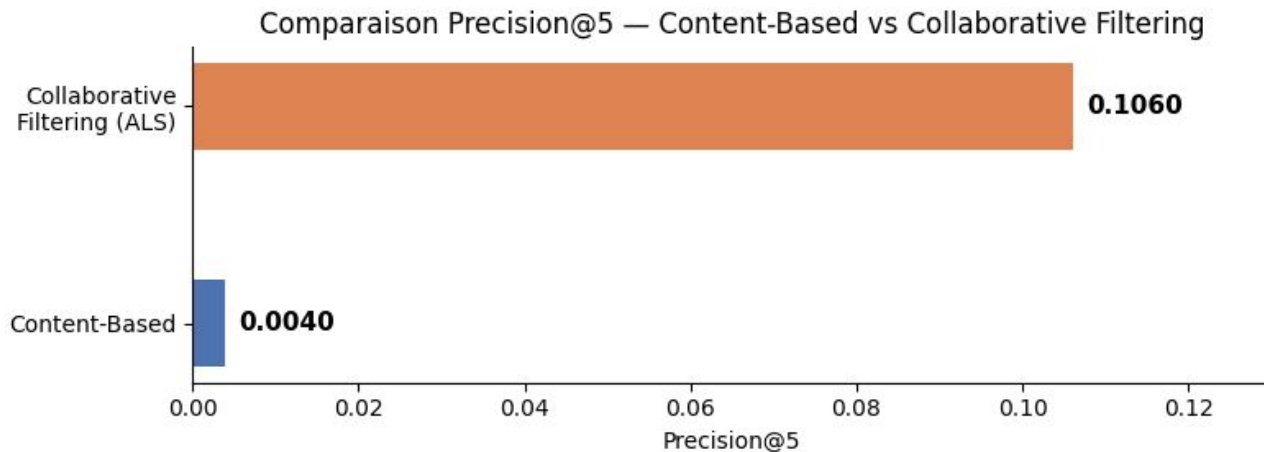
facteurs latents

ANALYSE

- Sur 500 utilisateurs testés, l'article masqué apparaît dans le Top 5 dans **10,6 %** des cas
- **26 fois plus performant** que le Content-Based sur ce jeu de données
- **Temps d'évaluation long** (~355 min) : le leave-one-out nécessite un réentraînement partiel par utilisateur — acceptable pour l'évaluation mais non viable en production

04-A. COMPARAISON QUANTITATIVE

CRITÈRES	CONTENT-BASED	COLLABORATIVE FILTERING
HitRate@5	0,40%	10,60%
Temps d'évaluation	-5 min	-355 min
Taille des artefacts	72.8 Mo	147,6 Mo
Explicabilité	94.53% de variance conservée ACP	50 facteurs latents (boîte noire)



04-B. COMPARAISON QUALITATIVE

CRITÈRES	CONTENT-BASED	COLLABORATIVE FILTERING
Cold-start utilisateur	Dès 1 clic	Historique requis
Cold-start article	Géré (embedding disponible)	Non géré
Ajout de nouveaux articles	<code>pca.transform()</code> suffit	Réentraînement complet
Ajout de nouveaux utilisateurs	Pas de réentraînement	Réentraînement complet
Interprétabilité	Bonne (similarité sémantique)	Faible (facteurs latents)
Temps d'entraînement	Rapide	Long (ALS itératif)

05- CHOIX RETENU & STRATÉGIE POUR LE MVP

MODÈLE RETENU : LE COLLABORATIVE FILTERING (ALS)

- **Hit Rate@5 26 fois supérieure au CB (0.1060 vs 0.0040)** > la **qualité de recommandation** prime pour valider un MVP
- Exploiter aux mieux les comportements collectifs de **322 897 utilisateurs**

LES LIMITES DU CF SONT CONNUES ET GÉRÉES

- Cold-start utilisateur > fallback top 5 articles populaires (article #1 : 37 213 clics)
- Cold-start article > CB dans l'architecture cible
- Interprétabilité limitée > acceptable pour un MVP

VISION PRODUCTION : ARCHITECTURE HYBRIDE

- Utilisateur avec historique > CF (ALS)
- Nouvel utilisateur, quelques clics (pas encore dans l'ALS) > Content-Based
- Nouvel utilisateur, 0 clic > Fallback popularité
- Nouvel article > embedding calculé + PCA > disponible immédiatement via CB, intégré à l'ALS au prochain réentraînement

06- DESCRIPTION FONCTIONNELLE DE L'APPLICATION

FLUX DE L'APPLICATION



DÉTAILS

- **Entrée** : identifiant utilisateur (entier) via paramètre URL ou body JSON
- **Traitement** : artifacts chargés une seule fois au démarrage (pas à chaque appel) > appel quasi-instantané
- **Sortie** : JSON avec user_id + liste de 5 article_id recommandés
- **Fallback** : utilisateur inconnu > top 5 articles les plus populaires automatiquement
- **Affichage** : pour chaque article > catégorie, nombre de mots, date de publication

07- DÉMO AVEC INTERFACE STREAMLIT

FONCTIONNALITÉS

- **Liste déroulante** des 200 premiers utilisateurs connus + 1 utilisateur fictif
- **Profil affiché** dès la sélection de l'utilisateur : nombre de clics et catégories déjà consultées
- **Bouton "Obtenir les recommandations"** → appel HTTP vers l'Azure Function
- **Affichage du Top 5 en cartes** : ID article, ID catégorie, nombre de mots, date de publication
- **Message contextuel** : fallback popularité si utilisateur inconnu, récapitulatif historique sinon

Système de recommandation d'articles

Sélectionnez un utilisateur pour obtenir ses 5 articles recommandés.

Identifiant utilisateur & historique en nombre de clics

Nouvel utilisateur — 0 clic (fallback)

Obtenir les recommandations

Utilisateur inconnu du modèle — affichage du fallback : top 5 articles les plus populaires

#1	Article 160974	·	Catégorie 281	·	259 mots	·	02/10/2017
#2	Article 272143	·	Catégorie 399	·	184 mots	·	02/10/2017
#3	Article 336221	·	Catégorie 437	·	158 mots	·	10/10/2017
#4	Article 234698	·	Catégorie 375	·	183 mots	·	10/10/2017
#5	Article 123909	·	Catégorie 250	·	240 mots	·	05/10/2017

07- DÉMO AVEC INTERFACE STREAMLIT - suite

Identifiant utilisateur & historique en nombre de clics

Nouvel utilisateur — 0 clic (fallback)

Nouvel utilisateur — 0 clic (fallback)

Utilisateur n°0

Utilisateur n°1

Utilisateur n°2

Utilisateur n°3

Identifiant utilisateur & historique en nombre de clics

Utilisateur n°122

Historique : 42 clic(s)

Catégories consultées : 118 · 134 · 186 · 209 · 250 · 281 · 289 · 299 · 301 · 323 · 331 · 375 · 388 · 399 · 409 · 412 · 421 · 431 · 435 · 437 · 442

Obtenir les recommandations

Top 5 articles recommandés pour l'utilisateur 122 :

#1 Article 284985 · 📁 Catégorie 412 · 📝 184 mots · 📅 07/10/2017

#2 Article 58580 · 📁 Catégorie 118 · 📝 138 mots · 📅 13/10/2017

08- ARCHITECTURE RETENUE – SCHÉMA

PHASE ENTRAÎNEMENT (LOCAL)

Notebooks Python

exploration, CB, CF, comparaison



Script d'entraînement

train_and_save_model.py



Azure Blob Storage

5 artifacts .pkl (als_model, matrices...)

PHASE INFÉRENCE (AZURE – CHARGEMENT AU DÉMARRAGE)

Azure Function

func-mycontent-reco, Python 3.12
Consumption plan



HTTP GET (user_id)



Application Streamlit

interface locale (MVP), Top 5 + métadonnées

09- COLD-START — PROBLÈME & SOLUTION ACTUELLE

17

POURQUOI LE COLD-START EST DÉTERMINANT POUR MY CONTENT

My Content est en phase de lancement => beaucoup de nouveaux utilisateurs attendus
L'architecture cible doit absolument prévoir ce cas dès la conception

COLD-START UTILISATEUR

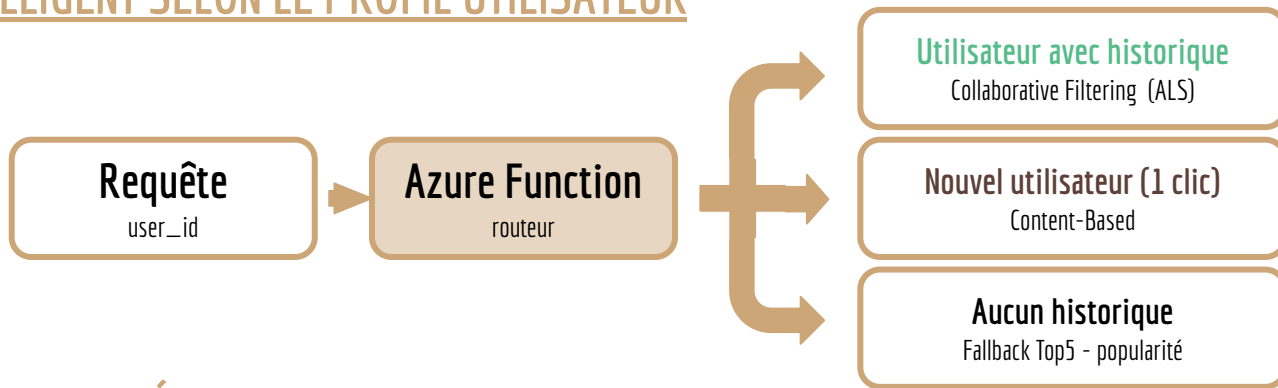
- Problème : un nouvel utilisateur sans historique de clics ne peut pas être traité par l'ALS
=> **Solution MVP** : fallback automatique sur les 5 articles les plus populaires

COLD-START ARTICLE

- Problème : un nouvel article jamais cliqué n'a pas de ligne dans la matrice — l'ALS ne peut pas le recommander
=> **Solution MVP** : non géré
=> **Solution CIBLE** : utiliser les embeddings (Content-Based) pour les nouveaux articles

10- ARCHITECTURE CIBLE — NOUVEAUX UTILISATEURS & ARTICLES

ROUTAGE INTELLIGENT SELON LE PROFIL UTILISATEUR



MISE À JOUR DES DONNÉES

- **Nouveaux articles** : calcul embedding => ACP (`pca.transform()`) => upload Blob Storage => disponible immédiatement pour le Content-Based, intégré à l'ALS au prochain réentraînement
- **Nouveaux utilisateurs** : CB dès le 1er clic, fallback à 0 clic — intégrés à l'ALS au prochain réentraînement (⚠ user_id central)

STRATÉGIE DE RÉENTRAÎNEMENT DE L'ALS

- Réentraînement **périodique** (ex. toutes les nuits ou toutes les semaines) : le plus courant en production
- Réentraînement **par seuil** : déclenché quand un volume significatif est atteint (ex. +5 % de nouveaux utilisateurs)
- Entre deux réentraînements, le **CB** prend le relais pour les nouveaux articles et utilisateurs — c'est précisément son rôle dans l'architecture hybride

11- INDUSTRIALISATION & MLOps

VERSIONING - GitHub

- **Dépôt** : Oxelie/P10_MyContent_recommandation
- **Branche dev** : développement local (Azurite)
- **Branche main** : production Azure (Blob Storage)

ENTRAÎNEMENT & DÉPLOIEMENT

- **Script train_and_save_model.py** : entraîne le modèle sur le dataset complet et sauvegarde les 5 artifacts
- Artifacts versionnés sur **Azure Blob Storage**
- **Déploiement** : `func azure functionapp publish`

STRUCTURE DU PROJET

- notebooks/
 - 01_exploration.ipynb
 - 02_content_based.ipynb
 - 03_collaborative_filtering.ipynb
 - 04_comparaison.ipynb
- scripts/
 - train_and_save_model.py
- azure_function/
 - function_app.py · requirements.txt
- app/
 - streamlit_app.py

12- BILAN & PERSPECTIVES

RÉALISATIONS DU MVP

- 2 approches testées et comparées (CB + CF)
- CF (ALS) avec Hit Rate@5 = 10,60%
- **Azure Function déployée et opérationnelle**
- **Interface Streamlit fonctionnelle**
- **Fallback cold-start implémenté**
- Code versionné sur **GitHub** (dev / main)
- **Validation technique d'un système de recommandation ServerLess** sur Azure

PERSPECTIVES DE PRODUCTION

- Architecture **hybride** CF + Content-Based
- Pipeline de réentraînement **automatisé** (Azure ML)
- **Collecte** de vraies données utilisateurs My Content
- **Authentification** Azure Function (FUNCTION level)
- **A/B testing** des approches en production
- **Métriques métier** : taux de clic, engagement (temps passé, articles lus)

12- ÉVOLUTIONS POSSIBLES

SUIVI DE LA DIVERSITÉ DES RECOMMANDATIONS

- **Problème** : si un utilisateur reçoit toujours les mêmes 5 articles > signal d'appauvrissement du modèle ou d'un biais de renforcement, 'bulle de filtre' qui s'installe progressivement
- **Solution** : tracer les recommandations produites par utilisateur et dans le temps > détecter les répétitions, mesurer la diversité (entropie, taux de renouvellement)

ADAPTATION DU COMPORTEMENT DE L'UTILISATEUR AUX RECOMMANDATIONS

- **Signal implicite** : l'utilisateur clique-t-il sur les articles recommandés ? Les lit-il en entier ?
- **Profil de curiosité** : certains utilisateurs cherchent à rester dans leur thème (bulle assumée), d'autres aiment découvrir — le modèle devrait s'adapter
- **Levier** : ajuster la pondération diversité/pertinence selon le profil observé (ex. paramètre α sur la similarité cosinus ou bruit exploratoire sur l'ALS)

PRISE EN COMPTE DE LA SAISONNALITÉ

- **Exemples concrets** : Coupe du Monde > pic d'articles sportifs ; fêtes de fin d'année > actualités culturelles/économiques ; élections > articles politiques
- **Risque sans gestion** : le modèle sur-recommande des articles populaires au moment de l'entraînement, même si la saison est passée
- **Solutions** : pondérer les clics récents plus fortement (decay temporel), réentraîner plus fréquemment en période de pic, ou ajouter un signal calendaire explicite

Annexe. DOCUMENTATION

Panorama des stratégies de recommandation

- [Recommender Systems Handbook](#)
- [The Netflix Recommender System](#)

Collaborative Filtering / ALS

- <https://github.com/benfred/implicit>
- ["Collaborative Filtering for Implicit Feedback Datasets" \(Hu, Koren, Volinsky 2008\)](#)

Collaborative Filtering / SVD

- [Recommender System made easy with Scikit-Surprise](#)

Content-Based / embeddings

- [Scikit-learn Décomposition PCA](#)
- [Scikit-learn Similarité cosinus](#)

Azure Functions / Serverless

- [Bindings Blob Storage](#)
- [Créer une application de fonction serverless avec Azure Fonctions](#)

Evaluation – Hit Rate@K

- [Evaluating Recommender Systems - Metrics for evaluating Recommender Systems](#)
- [Ranking Evaluation Metrics for Recommender Systems](#)



Merci de votre attention

